ConsentShield — Architecture Brief

Technical, security, and compliance architecture for evaluation by engineering, security, and compliance teams

ConsentShield · Hyderabad, India

April 2026 · v1.0

Table of Contents

Architecture Brief

For: CTOs, CISOs, and Data Protection Officers evaluating ConsentShield **Scope:** Standalone — contains complete integration contracts, security posture, and due-diligence answers. No supporting documents required. **Confidentiality:** For prospective customer review. Contains no production secrets, no specific infrastructure identifiers, no third-party credential material.

Document structure

Part I is an executive overview, readable in 20 minutes, suitable for all three audiences. Part II is a deep appendix for vendor due diligence, DPIA work, and legal-counsel sign-off.

One section — **Integration depth** (Section 6) — sits deliberately in Part I because it is the question every technical buyer asks inside the first meeting: *"what does integrating this actually require from my team?"* Section 6 defines the four integration depths at an architectural level. The specific API contracts, SDK patterns, webhook protocols, and connector catalogue behind them are in Section 11 (Integration contracts), in Part II.

Part I — Executive Overview

1. The product in one paragraph

ConsentShield is a DPDP compliance enforcement engine. It does three jobs. It **collects** consent as DEPA-native artefacts — one per purpose, independently revocable, with defined scope and expiry. It **enforces** that consent by monitoring third-party scripts on customer web properties in real time, revoking artefacts on withdrawal, and orchestrating deletion across connected systems with signed receipts. It **proves** all of this to the Data Protection Board (or any auditor) through a full chain-of-custody export — written to storage the customer controls, not storage ConsentShield holds.

Everything else in this document flows from the third item. The canonical compliance record — the audit evidence that matters legally — lives with the customer, not with ConsentShield. This is a deliberate architectural choice with material consequences for risk allocation, breach exposure, and contractual posture. It is not a marketing claim.

2. Architectural identity — stateless compliance oracle

ConsentShield operates as a **stateless compliance oracle**. Three principles follow directly from that identity and govern every design decision.

Principle one — process, deliver, delete. Every piece of Data Principal data that enters ConsentShield exits to customer-controlled storage within minutes. The platform's buffer tables are write-ahead logs, not databases. A row that has been delivered and confirmed has no reason to persist; it is deleted immediately, not on a schedule.

Principle two — the customer is the system of record. Dashboard views may read from short-lived buffers for real-time display. Compliance exports, audit packages, and any Data Protection Board-facing artefact are either read from — or direct the reader to — customer-owned storage.

Principle three — ConsentShield is a Processor, not a Fiduciary. Under the DPDP Act, a Fiduciary faces per-violation penalties of up to ₹250 crore. A Processor that accumulates a centralised record of everything it processes starts to resemble a Fiduciary. The stateless oracle architecture ensures ConsentShield does not cross that line.

The product description this produces is not a marketing claim — it is a description of how the system works: *"ConsentShield generates the compliance record and delivers it to the customer. We cannot be compelled to produce what we do not have. If ConsentShield shuts down tomorrow, the complete audit trail is already in the customer's storage, readable without ConsentShield."*

3. Stack at a glance

Modern, boring, well-understood stack. Deliberately so — this is compliance infrastructure, not a platform for speculative engineering.

Layer	Technology	Role
Frontend application	Next.js on Vercel · TypeScript · Tailwind · shadcn/ui	Customer dashboard, configuration, workflow UI
Identity	Supabase Auth	Email, magic link, Google OAuth, SSO
Operational database	Supabase Postgres with Row-Level Security	Organisation config, buffers, operational state
Edge runtime	Cloudflare Workers + KV	Banner delivery, consent event ingestion, tracker observation ingestion
Async workers	Supabase Edge Functions	Delivery orchestration, SLA

Layer	Technology	Role
	(Deno)	timers, security scans, deletion dispatch
Default storage	Cloudflare R2	Customer-owned canonical record (per-customer keys)
Bring-your-own storage	AWS S3 or Cloudflare R2	Customer-provisioned bucket; write-only credential
Billing	Razorpay Subscriptions	INR monthly and annual plans
Email	Resend	Transactional email
Error monitoring	Sentry	De-identified application errors

The design decision worth flagging to a security reviewer: Supabase Auth and Supabase Postgres are the same system. Policy functions are available inside every Row-Level Security policy. Multi-tenant isolation is enforced at the database layer, not in application code. Every query passes the RLS policy — there is no code path in which isolation can be forgotten.

4. Data classification — the operational / buffer / health split

Every table in ConsentShield's database belongs to exactly one of three categories. This is the most important thing to understand before evaluating data-protection posture.

Category A — Operational state (permanent). Data ConsentShield needs to function: organisation configurations, banner settings, billing records, team membership, tracker signature definitions, sector templates. Standard B2B SaaS business data. Retained for the life of the customer relationship.

Category B — User data buffer (transient). Data Principal personal data flowing through ConsentShield on its way to customer-controlled storage: consent events, audit log entries, tracker observations, deletion receipts, processing log entries, security scan results, withdrawal verification results. **Buffered for delivery, then deleted immediately** on confirmed customer-side write.

Category C — Zero persistence. FHIR records from ABDM integrations, and — for BFSI customers — PAN values, Aadhaar values, bank account numbers, balances, transactions, repayment history, and bureau pulls. **Never** written to any table, log, or file. Flows through ConsentShield's memory only, processed (consent gating, metadata extraction, drug-interaction check), then released. Any code path that attempts to persist Category C content is rejected in review without exception.

The practical consequence: the total quantity of Data Principal personal data resident in ConsentShield's systems at any given moment is small and bounded. In-flight events awaiting delivery — typically seconds to minutes of throughput. Any buffer row over one hour old triggers investigation; any row over 24 hours is a P0 incident.

5. DEPA-native consent model

Every India-focused competitor uses a GDPR-adapted consent model: a single event row with an array of accepted purpose labels. This works for cookie banners. It does not satisfy DEPA's structural requirement that *each data flow is authorised by a discrete, independently revocable, time-bounded, machine-readable artefact.*

ConsentShield's consent model is DEPA-native from the first schema row. The interaction log records the user's action; each event spawns N artefact records — one per accepted purpose. Each artefact has:

- **Artefact scope** — one purpose per artefact. Analytics, marketing, and personalisation are separately addressable.
- **Data scope** — the specific data fields this artefact authorises, drawn from the organisation's Purpose Definition Registry.
- **Time bounds** — explicit expiry. Consent is not open-ended.
- **Revocation chain** — revocation is an immutable row referencing the specific artefact, not an update to the original consent record.
- **Chain of custody** — the deletion requests a revocation triggers, and the deletion receipts that confirmed completion, are linked to the artefact.

An auditor can trace from consent grant to data deletion in a single query. This is the architectural moat. A GDPR-adapted tool cannot catch up without a schema rewrite.

6. Integration depth — what to build on your side

This is the decisive question for most technical evaluations: *"what does integrating ConsentShield actually require?"* The answer depends on how deeply the platform needs to be embedded. Four integration tiers exist, each with a well-defined scope, architectural implication, and typical effort profile. Customers move through them as their compliance surface widens, not all at once.

This section covers what each tier *is*, what it implies architecturally, and what it takes to ship. The specific API payloads, SDK snippets, webhook contracts, signature protocols, and the connector catalogue are in **Section 11 (Integration contracts)** in Part II.

6.1 The four integration surfaces

Surface	Direction	Purpose	Typical effort
Surface 1 — Consent capture	Customer → ConsentShield	Produce DPDP-compliant consent artefacts at the point of collection	10 minutes (web banner) · 1–2 days (custom UI via API)
Surface 2 — Consent verification	Customer → ConsentShield	Server-side check that a valid artefact exists before acting on data	1–3 days per calling system
Surface 3 —	ConsentShield →	Propagate revocation	1 hour (pre-built

Surface	Direction	Purpose	Typical effort
Deletion orchestration	Customer's downstream systems	and erasure with field-level precision, with signed receipts	OAuth) to 2 weeks (custom webhook connector)
Surface 4 — Operational notifications	ConsentShield → Customer's ops channel	Alerts on violations, rights requests, SLA breaches	15 minutes

6.2 The four integration depths

Surfaces are building blocks. Depths are the archetypal shapes customer integrations take. Every ConsentShield customer lives at one of these four depths at any given time.

Depth	Typical customer	Surfaces needed	Processing mode	Effort to live	Ongoing maintenance
D1 — Script tag	SaaS startup, D2C brand, single-page marketing site	Surface 1 only	Standard	10 minutes	None — the snippet self-updates
D2 — App-embedded	Multi-product SaaS, edtech platform, mobile-web app	Surface 1 + 2 + 4	Standard or Insulated	1–2 days	Light — consent-gated APIs added alongside new features
D3 — Back-office integrated	D2C operator with marketing stack, mid-market SaaS with CRM / CDP	Surface 1 + 2 + 3 + 4	Insulated	1–2 weeks	Moderate — connectors maintained as downstream APIs change
D4 — Full enterprise embedding	BFSI (NBFC, broking, SFB), hospital chain, enterprise with regulated third-party sharing	All 4, with custom connectors	Insulated or Zero-Storage	4–8 weeks	Non-trivial — ongoing once operational, but the cost is in orchestration, not ConsentShield per se

D1 — Script tag ("we only need a banner")

The integration is a single `<script>` tag pasted into the site's `<head>`. The banner renders, captures consent as DEPA-native artefacts, and monitors third-party scripts on the page. Consent events and tracker observations post to `cdn.consentshield.in` over HTTPS. The dashboard shows enforcement reports.

What the customer builds: nothing. Configuration is no-code in the dashboard — purposes, retention windows, banner copy, sector template.

Architectural implication: ConsentShield never sees any data from the customer's back-end systems. The only Data Principal data that passes through ConsentShield is what the browser sends during consent capture and tracker observation. Storage mode defaults to Standard. The customer is never exposed to more than the risk of consent records themselves — which are written to a ConsentShield-provisioned R2 bucket under a per-customer key delivered once and discarded on the ConsentShield side.

Who fits here: anyone whose regulated data flow starts and ends with web-tracking consent. Most Indian SaaS startups in 2026 fit this description.

D2 — App-embedded ("verify consent before we act on it")

The banner from D1 is in place. Additionally, the customer's application server performs a consent check before processing data. Before sending a marketing email, before running an analytics query that joins personally identifying fields, before pushing an event to CleverTap — the back-end asks ConsentShield *"is there a valid artefact for {user}, {purpose} right now?"* and proceeds only if yes.

What the customer builds: one helper library call in each code path that acts on personal data. Idiomatic wrapping of a single HTTPS call to the consent verification endpoint, with caching — ConsentShield returns an artefact validity summary that is safe to cache for a few minutes.

Architectural implication: still no ConsentShield access to customer data. The verification call carries only the Data Principal's identifier (email hash or internal user ID) and the purpose code. ConsentShield returns a pass/fail plus artefact expiry. Storage mode typically Insulated at this depth — the customer provisions their own R2 or S3 bucket for the canonical record.

Who fits here: multi-product SaaS firms, edtech platforms where children's-data obligations require demonstrable consent gating, mobile-web apps where back-end processing decisions need to be auditable.

D3 — Back-office integrated ("when they withdraw, it has to actually stop")

D1 and D2 are in place. In addition, when a Data Principal withdraws consent or makes an erasure request, ConsentShield orchestrates deletion across the customer's marketing, analytics, and support stack. Pre-built OAuth connectors cover Mailchimp, HubSpot, Zoho CRM, Freshdesk, Intercom, CleverTap, WebEngage, MoEngage, Shopify, WooCommerce, Razorpay, Segment. Each connector call returns a signed receipt that attaches to the revoked artefact.

What the customer builds: OAuth consents for each connected service (one-time, in the dashboard). For systems not in the pre-built connector catalogue, a generic webhook endpoint that accepts a deletion instruction and posts back a signed receipt — a few hundred lines of code per bespoke system.

Architectural implication: the customer's downstream systems — CRM, marketing platform, analytics warehouse — receive deletion instructions *scoped to the specific artefact that was revoked*. Marketing deletion does not touch bureau reporting; analytics revocation does not affect billing records. Insulated mode is required. The canonical deletion-receipt ledger lives in the customer's bucket.

Who fits here: D2C operators with material marketing spend, mid-market SaaS with 10+ connected systems, edtech with multiple engagement platforms, any business where withdrawal of consent has to propagate beyond the customer's own database.

D4 — Full enterprise embedding ("regulated third-party sharing, dual regimes, statutory exemptions")

All four surfaces, with custom connectors built against the customer's specific downstream systems. For BFSI customers: bancassurance partner APIs, co-lending fintech APIs, credit bureau APIs, CKYC platform. For healthcare: ABDM production integration, HIP APIs, ABHA-gated record retrieval. The Regulatory Exemption Engine is enabled — artefact revocation respects statutory retention requirements (RBI KYC retention, PMLA, SEBI LODR) while still discharging DPDP obligations precisely.

What the customer builds: a meaningful integration project, typically 4–8 weeks with a shared architect between ConsentShield and the customer's engineering team. The work is in mapping the customer's existing data flows to DEPA-native artefact boundaries, building custom connectors for bespoke downstream systems, and configuring the Regulatory Exemption Engine to the customer's specific statutory profile.

Architectural implication: Insulated mode mandatory. Zero-Storage mode mandatory for any FHIR data path. ConsentShield becomes the consent and deletion orchestration layer for the customer's regulated data; the customer's core systems remain the systems of record for the regulated data itself.

Who fits here: digital NBFCs and broking platforms, Small Finance Banks, digital arms of large private banks, hospital chains operating under ABDM, regulated enterprises with third-party sharing obligations (bancassurance, group-account consent, nominee and guarantor flows).

6.3 Choosing the right depth

The wrong choice is to over-integrate. A SaaS startup on Series A does not need D4 — it needs D1, shipping before the next board meeting. Equally, a BFSI customer that buys only D1 is not getting ConsentShield's value — it is getting a cookie banner.

A pragmatic selection rule:

1. If the customer's regulated data never leaves the browser and the back-end, D1 is sufficient.

2. If the customer's back-end acts on personal data (sending emails, computing segments, pushing to marketing platforms), D2 is the floor.
3. If consent withdrawal has to propagate to external systems to actually stop, D3 is the floor.
4. If the customer operates under dual regulatory regimes (DPDP plus RBI, SEBI, PMLA, or ABDM), D4 is the floor.

A customer can start at D1 and grow into D2, D3, and D4 as their compliance surface matures. Each depth is strictly additive; moving up a tier does not require re-architecting the tiers beneath it. The consent artefacts captured at D1 are the same artefacts that D4 revokes and orchestrates deletion against.

6.4 What ConsentShield does *not* require from the customer

Equally important for a technical evaluation — these are things other compliance platforms demand that ConsentShield deliberately does not:

- **No customer database access.** ConsentShield never reads from the customer's database. Not at D1, not at D4. The only inbound data paths are consent events and explicit API calls the customer's code initiates.
- **No customer application hosting.** ConsentShield runs on Vercel and Cloudflare. The customer's application runs wherever the customer hosts it. The integration is API-level, not deployment-level.
- **No customer source code access.** Pre-built connectors use public OAuth flows. Custom connectors are built from the customer's API documentation, not from code review.
- **No agent or sidecar deployment.** No binary runs in the customer's environment. No daemon, no container, no VPC peering, no AWS PrivateLink, no Kafka topic bridging.
- **No customer employee accounts in ConsentShield for Data Principals.** Data Principals interact with the customer's own website or app, not with ConsentShield. ConsentShield is invisible to them.

7. Processing modes — three postures for three risk profiles

The `storage_mode` on the organisations record determines the data-handling path. This check runs at the API gateway before any data write.

Mode	What ConsentShield holds	Customer storage	Who manages storage	Minimum tier
Standard	Operational config + encrypted buffer	ConsentShield-provisioned R2; per-customer key (delivered once, discarded)	ConsentShield provisions; customer holds key	Starter
Insulated (default at	Operational config only;	Customer's own R2 or S3 bucket;	Customer manages	Growth

Mode	What ConsentShield holds	Customer storage	Who manages storage	Minimum tier
Growth+)	short buffer	write-only credential		
Zero-Storage	Consent artefact index (TTL) + seconds-long delivery buffer	Customer's own bucket; data flows through memory only	Customer manages	BFSI / Healthcare

Zero-Storage is **mandatory** for health data (FHIR). Insulated is the default for Growth tier and above. Standard is for Starter-tier customers who cannot provision their own bucket.

Processing mode is orthogonal to integration depth, but typical pairings are: D1 with Standard, D2–D3 with Insulated, D4 with Insulated or Zero-Storage depending on the data classification.

8. Security posture — what matters for vendor review

The full technical and organisational measures list is in Annex 2 of the Data Processing Agreement. The architecturally-load-bearing claims — the ones that would change the vendor-risk rating if untrue — are:

Encryption. TLS 1.3 in transit. AES-256 at rest for all buffer tables and operational state. In Insulated and Zero-Storage modes, the customer holds their own storage encryption key; ConsentShield has write-only credentials against buckets it cannot read, list, or delete from.

Multi-tenant isolation at the database layer. Every Category A and B table is protected by Row-Level Security policies that enforce `org_id` matching on every query. The service role key (which bypasses RLS) lives only in server-side environment variables and the Cloudflare Worker. It is never in client code, never in logs, never in any audit trail. Belt-and-braces isolation: the API layer also validates `org_id` against the session JWT before issuing the query.

Buffer tables are append-only from authenticated users' perspective. No UPDATE or DELETE policy exists on consent events, tracker observations, audit log, or rights request events for any authenticated role. Writes come exclusively from the service role. Delivered rows are deleted by the service role immediately on confirmed delivery. A compliance record cannot be tampered with post-hoc because the write path is closed.

Rate limiting on public endpoints. The rights request submission endpoint (which accepts public-facing requests from Data Principals) is rate-limited at 5 requests per IP per hour, with the `org_id` taken from the URL path rather than the client payload.

Signature verification on billing webhooks. Razorpay webhooks are HMAC-SHA256 verified before any plan mutation is accepted.

Nightly security-posture scans. Each customer's web properties are scanned for SSL validity, HSTS, CSP, X-Frame-Options, vulnerable JavaScript libraries (version check against CVE database), mixed content, and cookie security flags. Findings surface in the compliance dashboard with severity gradations.

Incident response. 48-hour Security Incident notification to the customer under the DPA. Assistance with the customer's 72-hour DPDP Section 8(6) notification and, where applicable, Articles 33–34 GDPR notification. For BFSI customers, the dual-timeline breach workflow covers both the 6-hour RBI obligation and the 72-hour DPDP obligation from a single incident record.

Audit assurance. Annual external penetration test by a qualified third party. Continuous automated vulnerability scanning of dependencies. Quarterly internal access-control review. Public vulnerability disclosure programme with defined response SLA. Third-party audit reports (SOC 2, ISO 27001) provided on request where current.

9. What compliance teams care about — the DPO view

Two questions determine whether a DPO approves ConsentShield. This section answers both directly.

Question 1 — Who is the Data Fiduciary, who is the Data Processor, and is that split architecturally defensible?

The customer is the Data Fiduciary for all Data Principal personal data processed through the Service. ConsentShield is the Data Processor. Sub-processors (Supabase, Cloudflare, Razorpay, Resend, Sentry, and AWS for BYOS customers) are sub-contractors of ConsentShield.

The architectural claim that makes this split defensible rather than merely asserted: ConsentShield does not accumulate a centralised record of Data Principal data. The canonical compliance record lives in customer-controlled storage. The buffer tables hold in-flight events for seconds to minutes. At no point is ConsentShield holding "the customer's compliance database" — the customer is. This keeps ConsentShield firmly on the Processor side of the Section 8 analysis and materially reduces the customer's sub-processor risk.

Question 2 — On the day the Data Protection Board asks for evidence, what does the customer hand over?

The customer hands over an audit export generated from *their own* storage bucket, containing the full artefact register for the relevant time period: every consent grant, every revocation, every deletion request, every deletion receipt, every tracker observation, every violation, every rights request, every SLA event. Chain of custody from consent grant to deletion receipt, traceable in a single query per Data Principal.

The customer does not need to ask ConsentShield for this data. The customer does not need to file a subject-access request against ConsentShield. The customer does not need to preserve ConsentShield's logs. The evidence is the customer's own property in the customer's own storage — because that is where ConsentShield delivered it as soon as it was generated.

10. Limits of the platform — what ConsentShield does not do

A clear statement of scope limits is more useful than a long capabilities list.

ConsentShield is software, not legal advice. All templates — privacy notices, DPAs, sub-processor lists, sector templates — carry prominent disclaimers and should be reviewed by the

customer's legal counsel before deployment. Compliance outcomes are the customer's responsibility. The platform provides the infrastructure that makes compliance achievable and demonstrable; it does not *constitute* compliance.

ConsentShield does not host the customer's applications. The customer's website, app, and back-end run wherever the customer hosts them. ConsentShield is an API-level integration, not a deployment-level one.

ConsentShield does not manage the customer's marketing stack. Deletion orchestration revokes access; it does not curate campaigns, deduplicate lists, or make marketing decisions. Those remain the customer's functions.

ConsentShield does not adjudicate rights requests. The platform surfaces rights requests, tracks SLAs, supplies dashboard tooling, and orchestrates deletion. The legal decision of what to do with a given request — verify identity, approve erasure, partially respond, or reject — is the customer's Data Fiduciary function.

ConsentShield does not replace a Data Protection Officer. The DPO-as-a-Service marketplace is a referral to empanelled third-party DPOs who carry their own professional-advisory liability. ConsentShield carries software liability only, capped per the DPA.

No native mobile SDK in the current roadmap. Mobile-web consent capture works via the web banner. Native iOS/Android SDK is a 2027 roadmap item; customers building native apps today can integrate at the API level (Surface 1 via direct API call) instead.

Part II — Technical Appendix

11. Integration contracts — surfaces, payloads, and connectors

Section 6 described the four integration *depths* — the shapes a customer's integration takes. This section describes the four integration *surfaces* — the specific API contracts, payloads, and behaviours a depth is built from. Depths are combinations of surfaces; surfaces are the primitives.

Surface	Direction	Contract type	Detailed in
Surface 1 — Consent capture	Customer → ConsentShield	Banner script or REST API	§11.1
Surface 2 — Consent verification	Customer → ConsentShield	REST API (synchronous, cached)	§11.2
Surface 3 — Deletion orchestration	ConsentShield → Customer / downstream	OAuth or signed webhook	§11.3
Surface 4 — Operational notifications	ConsentShield → Customer's ops channel	Email, Slack/Teams webhook, custom webhook	§11.4

11.1 Surface 1 — Consent capture

Two modes. Every customer uses one or both.

Mode A — Web banner (script tag). The default for any web property.

```
<script src="https://cdn.consentshield.in/v1/banner.js"
        data-org="org_7H3K..."
        data-property="prop_BQ2X..."
        async></script>
```

Runtime behaviour: - Banner served from Cloudflare's edge in under 50 ms - User's decision is HMAC-signed client-side and POSTed to the Cloudflare Worker - Worker validates origin, HMAC signature, and payload; writes the event to the `consent_events` buffer; returns HTTP 202 - Fan-out pipeline creates one artefact per accepted purpose asynchronously - Banner continues monitoring for tracker violations after the consent decision (real-time enforcement) - On POST failure, decision is preserved in browser `localStorage`; retry reconciles on the next page load — a failed write must never break the user's browsing session

CMS variants for WordPress, Shopify, Webflow, Wix, Framer, and Squarespace are delivered as platform plugins. The payload and data model are identical — only the installation mechanism differs.

Mode B — Custom UI via Consent API. For mobile apps, call centres, kiosks, account-opening tablets, or any server-side recording of consent obtained through another channel.

```
POST https://api.consentshield.in/v1/consent/record
Authorization: Bearer cs_live_xxxxxxxxxxxxxxxx
Content-Type: application/json

{
  "property_id": "prop_mobile_BQ2X",
  "data_principal": {
    "identifier": "cust_987654",
    "identifier_type": "internal_customer_id"
  },
  "purposes": [
    { "purpose_definition_id": "pd_bureau_reporting_01HZY", "granted":
true },
    { "purpose_definition_id": "pd_insurance_marketing_01HZZ",
"granted": true },
    { "purpose_definition_id": "pd_whatsapp_marketing_01H00",
"granted": false }
  ],
  "captured_via": "mobile_app_onboarding",
  "captured_by": "system",
  "notice_version": "notice_v_2026_04",
  "captured_at": "2026-04-19T10:15:33Z"
}
```

ConsentShield validates that every `purpose_definition_id` exists in the Purpose Definition Registry (HTTP 422 if missing or invalid), writes the event, and returns one artefact ID per granted purpose. The customer stores these IDs against their account record in their own system.

Mobile-first customers (digital NBFCs, broking apps) use this mode. ConsentShield does not ship a native iOS/Android SDK today — the custom-UI-via-API pattern is the current integration model. A React Native SDK is under consideration for 2027; not a commitment.

Notice versioning. Every consent event records the `notice_version` the user saw at the moment of consent. When the customer updates their privacy notice, existing artefacts remain valid under the notice version they were captured against; new artefacts reference the new notice version. The audit export shows which notice version each artefact was executed against — essential for DPB defence of legacy consents.

11.2 Surface 2 — Consent verification

Every system in the customer's architecture that takes an action on behalf of a user should call this endpoint at the point of action.

```
GET https://api.consentshield.in/v1/consent/verify
    ?property_id=prop_core_banking
    &principal=cust_987654
    &purpose=insurance_marketing
Authorization: Bearer cs_live_xxxxxxxxxxxxxxxx
```

Response:

```
{
  "data_principal_identifier": "cust_987654",
  "purpose_code": "insurance_marketing",
  "status": "revoked",
  "active_artefact_id": null,
  "revoked_at": "2026-03-10T14:05:33Z",
  "revocation_record_id": "rev_01HXX7",
  "last_valid_artefact_id": "cs_art_01HXX2",
  "expires_at": null,
  "evaluated_at": "2026-04-19T10:15:33.445Z"
}
```

Sub-50 ms p99 latency, served from the validity cache. Safe to call synchronously from any server-side process.

Status field takes four values: - `granted` — an active non-expired artefact exists - `revoked` — the previous artefact was withdrawn by the user - `expired` — the previous artefact passed its expiry - `never_consented` — no artefact has ever existed for this principal/purpose pair

Customer code should treat the last three as functionally identical — *"do not act"* — and log which specific reason applied.

Where customers call this. A non-exhaustive list for a BFSI customer:

Customer system	When to call	Purpose code typically checked
Marketing campaign engine	Before adding a user to a campaign cohort	`whatsapp_marketing`, `email_marketing`, `sms_marketing`
Underwriting API (NBFC)	Before passing applicant's data to the scoring model	`credit_scoring`, `bureau_inquiry`
Insurance cross-sell batch job	Before sharing the daily data file with a bancassurance partner	`insurance_marketing`
Analytics ingestion layer	Before writing the event to Mixpanel / CleverTap	`analytics`
Push notification service	Before sending any non-transactional push	`push_marketing`
Contact-list sync (collections)	Before reading the user's phone contact list for recovery outreach	`contact_list_access`
Co-lending data share	Before the nightly partner reconciliation file is transmitted	`co_lending_partner`

Batch verification. For bulk operations — marketing segments, nightly partner files, bureau reporting:

```
POST https://api.consentshield.in/v1/consent/verify/batch
Authorization: Bearer cs_live_xxxxxxxxxxxxxxxx

{
  "property_id": "prop_core_banking",
  "purpose_code": "insurance_marketing",
  "data_principal_identifiers": [
    { "identifier": "cust_987654", "type": "internal_customer_id" },
    { "identifier": "cust_987655", "type": "internal_customer_id" }
  ]
}
```

Up to 10,000 identifiers per call. Parallel calls sustain higher throughput — the validity cache has ample headroom for bank-scale reconciliation runs.

Failure-mode handling. Three behaviours the customer's integration must plan for:

Response	Customer system behaviour	Rationale
`status: granted`	Proceed	Normal path
`status: revoked`, `expired`, or `never_consented`	Do not proceed; log with reason code	Normal path; the log is the audit trail

Response	Customer system behaviour	Rationale
API unreachable (timeout, 5xx, network error)	Do not proceed; log the failure; alert ops	**Fail-closed is the correct DPDP posture**

ConsentShield's client libraries (Node.js, Python, Java, Go — delivered at Pro and Enterprise tiers) default to a 2-second timeout and fail-closed. A customer who chooses to override — *"if ConsentShield is down, default to granted so business doesn't stop"* — is making an explicit compliance trade-off. We require an explicit named flag (`CONSENT_VERIFY_FAIL_OPEN = true`) that appears in the customer's audit export. The decision is visible.

Caching. Do not put your own cache in front of the verify endpoint. A withdrawal at 14:05:33 must invalidate the verification response at 14:05:34, not five minutes later after your cache TTL. The internal validity cache is updated by the revocation trigger before the transaction commits — the first verify call after revocation returns the revoked status. For higher throughput, use the batch endpoint.

11.3 Surface 3 — Deletion orchestration

When a Data Principal revokes an artefact, deletion is **artefact-scoped** — it fires against the fields in that artefact's declared `data_scope`, routed to connectors mapped to that artefact's purpose, for that specific Data Principal. This is the single most important design decision on Surface 3. It is not a blanket "delete everything about this user" sweep.

Two models for executing deletions, which customers mix and match per downstream system.

Model A — Pre-built OAuth connectors. For well-known SaaS tools with documented deletion APIs.

Setup: 1. Customer clicks Connect on the target service in the dashboard 2. OAuth redirect to the service's authorisation page 3. Customer authorises with delete-user scope 4. OAuth token stored encrypted in ConsentShield's vault, scoped to that organisation 5. Connector is active

Runtime: when a deletion is orchestrated, ConsentShield calls the service's API directly (e.g., `DELETE /lists/{id}/members/{hash}`) and records the response. No webhook on the customer's side.

Pre-built connector catalogue (April 2026):

Service	Category	Deletion operation	Status
Mailchimp	Email marketing	`DELETE /lists/{id}/members/{hash}`	Shipping
HubSpot	CRM	`DELETE /crm/v3/objects/contacts/{id}`	Shipping
Freshdesk	Support	`PUT /api/v2/contacts/{id}` (anonymise)	Shipping
Intercom	Support	`POST`	Shipping

Service	Category	Deletion operation	Status
		`/user_delete_requests`	
CleverTap	Engagement	`POST /delete/profiles`	Shipping
WebEngage	Engagement	`DELETE /users/{id}`	Shipping
MoEngage	Engagement	`DELETE /v1/customer/{id}`	Shipping
Shopify	E-commerce	`DELETE /customers/{id}`	Shipping
WooCommerce	E-commerce	`POST /customers/{id}/anonymize`	Shipping
Razorpay	Payments	`POST /customers/{id}/anonymize`	Shipping
Segment	CDP	`POST /regulations`	Shipping
Zoho CRM	CRM	`DELETE /crm/v2/Contacts/{id}`	Q3 2026
Freshworks CRM	CRM	`DELETE /contacts/{id}`	Q3 2026
Zendesk	Support	`POST /api/v2/users/{id}/deletions`	Q3 2026
Campaign Monitor	Email marketing	`DELETE /subscribers.json`	Q3 2026
Mixpanel	Analytics	`POST /api/2.0/gdpr-requests`	Q3 2026

Per-customer integration effort: one click. New connectors (if a customer's service is not listed) are built on request — typically 2–3 engineer-days on the ConsentShield side, delivered as part of onboarding. Custom connectors for bank-specific partners (bancassurance APIs, co-lending fintech APIs, bureau APIs) are built as part of the BFSI Enterprise engagement.

Model B — Generic webhook protocol. For every other system — the customer's core banking platform, internal CRM, data warehouse, regulatory reporting systems, or any partner vendor without a pre-built connector. The customer implements one HTTP endpoint.

Instruction from ConsentShield:

```
POST https://customer-api.bank.in/privacy/deletion
Content-Type: application/json
X-ConsentShield-Signature: sha256=<HMAC of body with shared secret>

{
  "event": "deletion_request",
  "receipt_id": "rcpt_01HXX8",
  "artefact_id": "cs_art_01HXX2",
  "data_principal": {
    "identifier": "cust_987654",
    "identifier_type": "internal_customer_id"
  },
  "reason": "consent_revoked",
  "data_scope": ["email_address", "name", "dob", "account_type",
"nominee_name"],
  "purpose_code": "insurance_marketing",
  "callback_url":
"https://api.consentshield.in/v1/deletion-receipts/rcpt_01HXX8?
sig=<HMAC>",
  "deadline": "2026-04-09T14:05:33Z",
  "issued_at": "2026-03-10T14:05:35Z"
}
```

Reason codes are a fixed enumeration: `consent_revoked`, `consent_expired`, `erasure_request` (DPDP Section 13 rights request — sweeps all active artefacts for the Data Principal), `retention_expired`.

Customer's callback to ConsentShield when deletion is complete (the callback URL is already HMAC-signed; no additional customer signature needed):

```
POST https://api.consentshield.in/v1/deletion-receipts/rcpt_01HXX8?
sig=<HMAC>
Content-Type: application/json

{
  "receipt_id": "rcpt_01HXX8",
  "status": "completed",
  "records_deleted": 1,
  "fields_deleted": ["email_address", "name", "dob", "account_type",
"nominee_name"],
  "systems_affected": ["bancassurance_partner_prod",
"bancassurance_partner_replica"],
  "completed_at": "2026-03-10T14:07:12Z",
  "operator": "system_auto",
  "evidence_reference": "partner_ref_XYZ123"
}
```

Partial completion and statutory-retention responses are first-class:

```
{
  "receipt_id": "rcpt_01HXX8",
```

```
  "status": "partial",
  "records_deleted": 1,
  "fields_deleted": ["email_address", "name"],
  "fields_retained": ["dob", "account_type"],
  "retention_reason": "Required for pending insurance policy
underwriting — statutory",
  "retention_statute": "Insurance Act 1938 § 64VB",
  "completed_at": "2026-03-10T14:07:12Z"
}
```

ConsentShield normalises partial responses and surfaces them on the compliance dashboard. A DPB examiner sees exactly what was deleted, what was retained, and which statute compelled retention.

Security properties: - Every instruction carries an `X-ConsentShield-Signature` header: HMAC-SHA256 of the body with a shared secret established at connector setup. The customer's endpoint **must** verify the signature before processing. - The callback URL includes its own HMAC signature derived from the receipt ID. Invalid signatures are rejected with HTTP 401. - Deletion receipt IDs are single-use. A replayed callback is rejected with HTTP 409. - The `deadline` field is binding. If the callback is not received by the deadline, the receipt transitions to `overdue`, an alert fires, and the DPB-facing audit export flags the missed SLA.

Integration effort on the customer side: typically 1–2 engineer-days to define the route, verify the signature, enqueue a deletion job, execute the deletion across the customer's internal stores, and post the callback. The real effort is understanding where the data lives in the customer's architecture and how to delete it cleanly — bounded by the customer's own complexity, not by ConsentShield's API.

The three-link audit chain. Every artefact-triggered deletion produces a complete, auditable chain of custody:

```
consent_artefacts.artefact_id
    └── artefact_revocations.artefact_id (the revocation record)
         └── deletion_receipts.artefact_id (one row per connector
instruction)
```

Rights-portal erasure requests and retention-rule expiries produce two-link chains starting at `rights_requests` or `retention_rules`. In every case, an auditor can reconstruct which user consented, when they withdrew, which systems were instructed to delete which fields, and when each system confirmed. This is the DPDP Section 12 evidence trail, produced as a by-product of normal operation rather than as a separate reporting exercise.

11.4 Surface 4 — Operational notifications

Set up once during onboarding and forgotten thereafter.

Channel	Setup effort	Typical use
Email (Resend)	0 — default compliance contact on the org record	Compliance officer, DPO
Slack incoming webhook	5 minutes	Engineering on-call channel

Channel	Setup effort	Typical use
Microsoft Teams webhook	5 minutes	Compliance team channel
Discord webhook	5 minutes	Startup engineering channel
PagerDuty / OpsGenie	10 minutes (via custom webhook)	Production incident routing
Custom webhook	15 minutes	Customer's internal alerting system

Alert types — each independently routable per channel, with severity (info, warning, critical) mapped per channel so critical alerts can go to PagerDuty while daily summaries go to Slack:

- Tracker violation detected (consent declined but tracker fired anyway)
- New rights request received (access, correction, erasure, nomination)
- SLA warning — 7 days remaining on a rights request
- SLA overdue
- Consent withdrawal verification failure — tracker continued firing after revocation
- Security scan: new critical finding on a monitored property
- Retention period expired on a data category
- Deletion orchestration failure (receipt transitioned to `failed` or timed out to `overdue`)
- Consent probe failure
- Daily compliance score summary
- Orphan consent event (DEPA fan-out pipeline stuck)
- Artefact expiry warning (30 days before expiry, so re-consent can be planned)

12. Reference architectures

Four archetypes covering the spectrum from lightweight SaaS to full BFSI enterprise deployment. Each maps an integration depth to a specific customer shape.

12.1 Pure web SaaS (Starter or Growth tier) → Depth D1 or D2

```
[user's browser]
      │
      │  loads page
      ▼
[customer's web app (Vercel / AWS / GCP)]
      │
      │  <script src="cdn.consentshield.in/v1/banner.js">
      ▼
[ConsentShield CDN] ─── consent events ──► [ConsentShield (Standard
or Insulated)]
                                                  │
                                                  │  deletion
orchestration (D2 only)
                                                  ▼
```

```
                        [Mailchimp · HubSpot · Intercom]  ← pre-built
OAuth connectors
```

- Processing mode: **Standard** (Starter) or **Insulated** (Growth)
- Surfaces used: 1 (banner), optionally 3 (pre-built OAuth connectors at D2)
- Integration effort: 10 minutes (D1) to 1 day (D2)
- Compliance outcome: consent capture with artefact-per-purpose precision; automatic deletion across the top 3–5 SaaS tools the customer uses

12.2 Mobile-first digital NBFC (BFSI Growth tier) → Depth D4

```
[user's iOS / Android app — NBFC's own native UI]
        │
        │  Custom UI via API: POST /v1/consent/record
        ▼
[NBFC's mobile backend — AWS / on-prem]
        │                                       │
        │  Surface 2: consent verification      │
        ▼                                       ▼
[ConsentShield] ◄── verify before underwriting  [underwriting API]
       │
       │  Surface 3: artefact-scoped deletion
       ▼
[core lending system]       ← generic webhook
[CleverTap · MoEngage]      ← pre-built connectors
[collections partner]       ← generic webhook
[bureau: CIBIL / Experian]  ← does NOT receive deletion (statutory
exemption)
```

- Processing mode: **Insulated** (customer's own S3 bucket)
- Surfaces used: 1 (custom UI via API), 2 (verification on every lending decision), 3 (mix of webhooks and connectors), 4 (PagerDuty for SLA alerts)
- Integration effort: 2–3 weeks total including mobile app UX integration
- Compliance outcome: DPDP-compliant at the moment of Android runtime permission grant; statutory retention correctly handled for bureau data; contact-list collection backstop closed

12.3 Mid-sized D2C e-commerce operator (Growth or Pro tier) → Depth D3

```
[storefront: Shopify / WooCommerce + Webflow]
      │
      │  banner.js + Shopify app
      ▼
[ConsentShield (Insulated)]
      │
      │  Surface 2 verify before each campaign send
      │  Surface 3 artefact-scoped deletion
      ▼
[Mailchimp]         [WebEngage]        [Shopify customers API]
      │                   │                        │
```

```
└──────────────────────┴──────────────────────────┘
                ↑ pre-built connectors ↑
```

- Processing mode: **Insulated**
- Surfaces used: 1 (banner), 2 (verify before marketing sends), 3 (pre-built connectors for the entire marketing stack), 4 (Slack for ops)
- Integration effort: 1 week including connector setup and notice-versioning
- Compliance outcome: every marketing send is consent-gated; every withdrawal propagates to every connected marketing system within minutes with signed receipts

12.4 Private bank deployment (BFSI Enterprise tier) → Depth D4

```
[bank's retail internet banking + mobile app]
        │  banner.js (web)          │  Custom UI via API (mobile)
        ▼                           ▼
        ┌──────────────────────────┐
        │ ConsentShield Zero-Storage │
        └──────────────────────────┘
        │  Surface 2: verify        │  Surface 3: artefact-scoped
deletion
        ▼                           ▼
[core banking]              [bancassurance partner]    ← custom BFSI
connector
[underwriting engine]       [co-lending fintech]       ← custom BFSI
connector
[cross-sell batch jobs]     [contact-list collections] ← generic
webhook
[bureau reporting] ← does NOT receive deletion (CICRA Section 17
exemption)
```

- Processing mode: **Zero-Storage** (customer's own R2 bucket; consent artefact index only; buffer in seconds, not minutes)
- Surfaces used: All four, with custom connectors for bank-specific systems and Regulatory Exemption Engine configured to the bank's statutory profile
- Integration effort: 4–8 weeks with a shared architect between ConsentShield and the bank's engineering team
- Compliance outcome: DPDP-compliant consent capture across web and mobile; consent verification on every partner data share; artefact-scoped deletion respecting RBI KYC, CICRA, PMLA, and SEBI retention requirements; dual 6-hour RBI / 72-hour DPDP breach notification workflow

13. Zero-persistence for regulated content

This is the single most important architectural claim for any BFSI or healthcare buyer — architecturally load-bearing, and the single answer to every *"what happens if ConsentShield is compromised?"* question.

13.1 The claim

Content-layer data governed by sector-specific retention regulation is **never** written to any ConsentShield table, any log, any file, or any buffer. It flows through ConsentShield's server in memory only, if at all. This is a structural property of the schema, not a policy document.

Category	Source	Governing regulation	ConsentShield treatment
PAN values	BFSI customers	RBI KYC Master Directions	Never persisted
Aadhaar values and Aadhaar-derived references	BFSI customers	Aadhaar Act, RBI KYC	Never persisted
Bank account numbers	BFSI customers	RBI KYC, Banking Regulation Act	Never persisted
Account balances	BFSI customers	Banking Regulation Act	Never persisted
Bank statements	BFSI customers	RBI record retention	Never persisted
Repayment history	BFSI customers	Credit Information Companies Act	Never persisted
Transaction records	BFSI customers	PMLA, Banking Regulation Act	Never persisted
Bureau pulls (CIBIL, Experian, CRIF)	BFSI customers	Credit Information Companies Act	Never persisted
KYC documents	BFSI customers	PMLA, RBI KYC	Never persisted
FHIR clinical records	Healthcare customers	ABDM, DISHA	Never persisted
Diagnoses, medications, lab results, prescriptions, observations, imaging	Healthcare customers	ABDM, DISHA	Never persisted

Any future regulated sector's content — telecom call detail records, insurance claims content, education records — inherits this zero-persistence category by default.

13.2 The category-label vs content-value distinction

DEPA artefacts hold *category declarations*, never values:

```
consent_artefacts.data_scope = ['pan', 'aadhaar_ref', 'account_type',
'repayment_history']
        ↑                          ↑                ↑                ↑
      a LABEL                    a LABEL          a LABEL        a
LABEL
```

```
  (declares "this consent covers PAN-type data"; actual PAN value
'ABCDE1234F' is never stored)
```

The artefact tells the deletion orchestrator *which categories to propagate*. The actual values live in the customer's systems — core banking, CRM, insurance partner — which is precisely where the customer's Fiduciary obligations under DPDP require them to be.

13.3 Why this matters for procurement

Every regulated BFSI customer will ask some version of: *"If your platform is compromised, does the attacker gain access to our customers' PAN numbers, account numbers, or balances?"*

The answer is: **no, because those values do not exist in ConsentShield's database.** The attacker gains category labels, purpose definitions, artefact IDs, and timestamps — operational metadata. They do not gain content.

For the same reason, the RBI outsourcing-guideline analysis becomes tractable. The customer is not outsourcing data storage to ConsentShield because ConsentShield does not store their customers' personal data. The customer is outsourcing consent *processing* — the same relationship a bank has with a payment gateway. The bank holds the cards; the gateway processes the transactions.

This is why Zero-Storage mode is the natural deployment for BFSI Enterprise, and why it is mandatory for healthcare: in Zero-Storage, even the identifiers and metadata flow through ConsentShield in memory only, giving the customer a maximally defensible architectural posture.

14. Detailed data-flow diagrams

Four core flows. Each one is described as a sequence, with the key invariant that makes it correct.

14.1 Consent capture — web banner flow

```
Data Principal's browser
     │
     │  GET kuruvi.in/product
     ▼
Customer's website loads
     │
     │  <script src="cdn.consentshield.in/v1/banner.js?...">
     ▼
Cloudflare Worker serves banner.js (cached in KV, <50ms)
     │
     │  Banner renders, trackers held
     ▼
User makes decision (accept / reject / customise)
     │
     │  POST cdn.consentshield.in/v1/events
     ▼
Cloudflare Worker
```

```
     │  Validate payload
     │  Truncate IP, hash user-agent
     │  Write to consent_events buffer (service role)
     │  Fan out to consent_artefacts — one per accepted purpose
     │  Return 202 immediately
     ▼
Delivery Edge Function (async)
     │  Read undelivered rows
     │  Write to customer-controlled storage
     │  On confirmed write → DELETE buffer row
     ▼
Customer's bucket holds the canonical record
```

Invariant: the Data Principal's browsing experience never waits on ConsentShield. The 202 returns before the buffer write completes; the buffer write returns before customer-storage delivery completes. Failure anywhere downstream never breaks the user's page load.

14.2 Consent verification — back-end flow (D2 and above)

```
Customer's application server
     │
     │  About to send marketing email to Mrs. Sharma
     │  GET api.consentshield.in/v1/consent/verify
     │    ?principal=sha256(mrs.sharma@company.in)
     │    &purpose=marketing_email
     ▼
ConsentShield API
     │  Check active artefact cache (TTL indexed)
     │  If expired or revoked: return FAIL
     │  If active: return PASS + artefact expiry
     ▼
Customer's application
     │  If PASS → proceed
     │  If FAIL → do not send; log decision
```

Invariant: the verification call carries a hashed identifier and a purpose code. It does not carry Data Principal personal data. The response is a small JSON object safe to cache for up to five minutes per principal-purpose pair.

14.3 Artefact revocation and deletion orchestration (D3 and above)

```
Data Principal withdraws consent via preference centre
     │
     ▼
Customer's preference centre → ConsentShield API
     │  POST revoke artefact CA_01J2K4F7N9P4R
     ▼
ConsentShield
     │  Write artefact_revocations row (immutable)
     │  Invalidate active-artefact cache entry
     │  Enqueue deletion requests scoped to artefact's data_scope
     ▼
```

```
Deletion orchestration (Edge Function)
     │  Resolve connected integrations for this artefact's purpose
     │    → Mailchimp (marketing_email scope)
     │    → CleverTap (behavioural scope)
     │    → internal sessions DB (via webhook)
     │  Dispatch deletion instructions in parallel
    ▼
Each downstream system
     │  Perform deletion
     │  POST signed receipt to api.consentshield.in/v1/deletion-
receipts/{id}
    ▼
ConsentShield
     │  Attach receipt to artefact revocation record
     │  Deliver receipt to customer-controlled storage
    ▼
Withdrawal verification scans
     │  T + 15 min → scan customer's site for marketing trackers
     │  T + 1 hour → re-scan (catches cached scripts)
     │  T + 24 hours → re-scan (catches persistent violations)
```

Invariant: deletion scope is bounded by the revoked artefact's declared `data_scope`. Revoking the "marketing" artefact does not trigger deletion of billing records. Revoking the "analytics" artefact does not touch nominee consent records. Artefact-scoped deletion is what makes this defensible under both DPDP and sector-specific retention regimes.

14.4 Audit export — the DPB evidence path

```
Compliance manager clicks "Export for audit" in dashboard
     │
    ▼
ConsentShield API
     │  /api/orgs/{orgId}/audit/export
     │  Parameters: date range, principal filter (optional), purpose
filter (optional)
    ▼
Edge Function generates export package
     │  Reads from customer-controlled storage (not from ConsentShield
buffers)
     │  Compiles: artefact register + revocations + deletion receipts +
     │            tracker observations + rights request history +
breach notifications
     │  Signs with ConsentShield root key (verifiable offline)
     │  Writes signed export to customer bucket
    ▼
Dashboard shows download link (direct from customer bucket)
     │
     │  Compliance manager downloads the evidence package
```

Invariant: the export is read from customer storage, not from ConsentShield's operational database. If ConsentShield is offline, the customer still has the data; they only lack the export-

generation convenience. The signed export is verifiable offline against ConsentShield's published root key.

15. Buffer lifecycle — zero tolerance for stale data

The buffer tables (consent_events, tracker_observations, audit_log, processing_log, delivery_buffer) are write-ahead logs. A row's lifecycle is seconds to minutes, not hours.

Preferred path — immediate deletion: after the delivery Edge Function confirms a write to customer-controlled storage, the buffer row is marked `delivered_at = now()` and immediately hard-deleted in the same transaction. A consent event successfully delivered at 14:32:01 has no business existing in ConsentShield's database at 14:32:02.

Safety net — 15-minute sweep: a scheduled job runs every 15 minutes to catch orphaned rows (process crash between mark and delete, confirmed-delivery message received but delete failed). In normal operation, this sweep finds zero rows. If it finds any, it deletes them and flags for investigation.

Alert threshold — 1 hour: any row older than one hour without delivery represents a delivery-pipeline failure and triggers an operational alert.

Hard emergency — 24 hours: any row older than 24 hours is a P0 incident. The delivery pipeline has been broken for a full day, and this is paged directly to the engineering team.

This is a compliance-infrastructure design choice, not an engineering aesthetic. The stateless oracle identity depends on the buffer actually being transient.

16. Security rules that are architectural constraints, not feature decisions

Twelve rules govern the platform's security posture. They cannot be relaxed without rebuilding substantial parts of the product.

1. **The service-role key never touches the browser.** Lives in server-side environment variables and Cloudflare Worker bindings. Never in client code, never in logs, never in any audit trail.
2. **Buffer tables are append-only for authenticated users.** No UPDATE or DELETE RLS policy exists for consent events, tracker observations, audit log, processing log, or rights request events for any authenticated role. Writes exclusively via service role.
3. **Health data (ABDM) is never stored.** FHIR records flow through memory only. No schema, no table, no log ever holds clinical content.
4. **org_id is validated at two levels.** API routes check the session's org_id against the resource being requested. RLS policies enforce the same check at the database level. Both must pass.
5. **Razorpay webhooks are signature-verified before processing.** Rejected if the HMAC signature does not match.
6. **The public rights request endpoint is rate-limited.** 5 requests per IP per hour. `org_id` taken from URL path, not client payload.
7. **ConsentShield's database is operational state, not a compliance record store.** Any feature that treats buffer tables as the system of record is architecturally wrong.

8. **Export credentials are write-only and never logged.** The IAM credential permits PutObject only. Stored encrypted at rest.
9. **Processing modes are enforced at the API gateway.** The storage_mode check runs before any data write. A Zero-Storage organisation cannot have data written to any persistent table.
10. **RLS policies are the first code committed.** Schema design and RLS policy definitions are written and tested before any customer data exists. A consent log that leaks across tenants — even briefly — is a catastrophic trust event for a compliance product.
11. **Buffer rows do not persist after delivery.** Deletion is immediate on confirmed delivery.
12. **Integration connector credentials are encrypted at rest.** OAuth tokens for third-party deletion connectors are stored encrypted; never logged, never exported, never in audit packages.

17. Multi-tenant isolation — implementation depth

For reviewers who want to verify the isolation claims:

JWT custom claims. After signup and organisation creation, `org_id` and `org_role` are injected into every JWT via Supabase's custom access token hook. These claims are available in every RLS policy via the `auth.jwt()` function.

Helper functions. `current_org_id()` returns the org_id from the JWT. `is_org_admin()` returns true if the role is admin. These functions are `stable` and `language sql` — inlined by the planner, zero query-time overhead.

Three isolation patterns. Every table follows one of three patterns:

- *Pattern 1 — org-scoped read/write (operational tables):* SELECT, INSERT, UPDATE policies all checking `org_id = current_org_id()`.
- *Pattern 2 — org-scoped read-only (buffer tables):* only a SELECT policy for the authenticated user. No INSERT/UPDATE/DELETE policy for any authenticated role. Writes come exclusively from service role (which bypasses RLS).
- *Pattern 3 — public insert, org-scoped read (rights requests):* public INSERT (rate-limited at the API layer), authenticated SELECT/UPDATE scoped to org.

Defence in depth. The API layer validates `org_id` against the session JWT before issuing the query. The RLS policy re-validates at the database layer. Both must pass.

18. Sub-processor register — full list

Sub-processor	Role	Processing location	Bound by
Supabase Inc.	Authentication, operational Postgres database	Regional, customer-addressable	DPA with ConsentShield, SOC 2, GDPR Art. 28
Cloudflare Inc.	CDN, edge workers, default R2 object storage	Global edge network	DPA with ConsentShield, SOC 2, ISO 27001

Sub-processor	Role	Processing location	Bound by
Razorpay Software Pvt Ltd	Subscription billing and invoicing	India	DPA with ConsentShield, PCI-DSS
Resend Inc.	Transactional email	United States	DPA with ConsentShield
Sentry Inc.	Application error monitoring (de-identified)	United States	DPA with ConsentShield, SOC 2
Amazon Web Services Inc.	Optional BYOS (Bring Your Own Storage) — customer-selected S3	Customer-selected region	DPA with customer directly

Sub-processor change notifications are dispatched at least 30 days before onboarding a new sub-processor, with a 20-day objection window for the customer per DPA Section 5.

19. Incident response and breach timelines

Internal incident classification. P0 = data integrity or confidentiality incident, customer-affecting. P1 = service-affecting incident, no data impact. P2 = sub-system degradation. P3 = minor. P0 and P1 trigger the documented incident-response runbook including on-call engineer paging, executive notification, and customer communications planning.

Security Incident notification to customers (DPA Section 7). Confirmed Security Incident affecting a customer's Personal Data is notified to the customer within 48 hours of confirmation. The notification includes nature, categories and approximate numbers of Data Principals and records affected, likely consequences, measures taken or proposed, and the ConsentShield point-of-contact.

DPDP Section 8(6) — customer's 72-hour obligation. ConsentShield assists the customer with their 72-hour DPB notification by providing the forensic information required within the customer's timeline. For BFSI customers operating under RBI cyber-incident guidelines, the dual-timeline breach workflow covers both the 6-hour RBI obligation and the 72-hour DPDP obligation from a single incident record.

GDPR Articles 33–34. For EU Personal Data flows, ConsentShield assists the customer with the Article 33 notification (72-hour to supervisory authority) and Article 34 notification (to affected Data Subjects where required) via the Security Incident support in the DPA.

No direct regulator notification. ConsentShield will not notify regulators or Data Principals directly, except where required by law or where the customer has failed to do so after reasonable notice and the failure is likely to cause material harm to Data Principals.

20. Change management and architectural invariants

Three architectural invariants are treated as non-negotiable. A proposed change that violates any of them is rejected in design review without further consideration:

- **Stateless oracle identity.** No feature that treats ConsentShield's operational database as the system of record for Data Principal personal data.
- **Category C zero persistence.** No code path that writes FHIR content, PAN/Aadhaar values, bank account numbers, balances, transactions, or bureau pulls to any table, log, or file.
- **Multi-tenant isolation at the database layer.** No feature that relies on application-code checks for tenant isolation. RLS policies remain the ground truth.

Everything else — pricing, connectors, sector templates, notification channels, dashboard UX — is variable and evolves based on customer feedback. These three do not.

21. Due-diligence question bank — referenced answers

This is the matrix that most vendor-review forms convert to. Each answer is traceable back to a section of this document.

Question	Section	Short answer
Is the platform ISO 27001 or SOC 2 certified?	§8	Third-party audit reports available on request where current. Annual external penetration test.
How is tenant isolation enforced?	§14	Row-Level Security at the database layer, with API-layer validation as belt-and-braces.
Where is customer data stored?	§7	Processing mode determines it. Standard: CS-provisioned R2. Insulated/Zero-Storage: customer's own R2 or S3.
Can ConsentShield read our data once exported to our storage?	§7, §10	No. Write-only credential. No `GetObject`, no `ListObjects`, no `DeleteObject` permission.
What is the breach notification timeline?	§16	48 hours to customer under DPA. Supports customer's 72-hour DPDP and Article 33 GDPR obligations. BFSI dual-timeline covers the 6-hour RBI window.
Is there a mobile SDK?	§10	No native SDK in current roadmap. Mobile-web works via the standard banner. Native apps integrate at the API level.
Does ConsentShield require	§6.4	No. Not at any integration

Question	Section	Short answer
access to our production database?		depth.
How is data encrypted at rest?	§8	AES-256 for buffer tables and operational state. Customer-held key in Insulated and Zero-Storage modes.
What are the sub-processors?	§15	Supabase, Cloudflare, Razorpay, Resend, Sentry, AWS (BYOS only). Full register in §15.
Can we test without signing a DPA?	Contact	Yes — a read-only sandbox is provided for technical evaluation. Production use requires the DPA.
What is the liability cap?	DPA §8	Twelve months of fees paid, inclusive of indemnification. Unlimited Customer-side indemnity applies for Customer Data and misuse.
What happens if ConsentShield shuts down?	§2, §14.4	Customer retains full compliance record in own storage. Signed audit exports verifiable offline.
Is the platform compliant with DPDP Section 16 (cross-border)?	§15	Primary infrastructure in India. Sub-processor edge locations comply with DPDP transfer restrictions. No processing in jurisdictions notified as restricted.

This Architecture Brief is version 1.0, prepared April 2026. For questions specific to a procurement conversation, contact the ConsentShield team at hello@consentshield.in. For detailed integration contracts, API payloads, and connector catalogues, see the Customer Integration Whitepaper v2.1.